TCS IPA COMPREHENSIVE MASTER PACK

TCS IPA 35-Mark Java Practice Master Handbook

Structured Solutions: Business Functions First → Main Runner → Clean POJO Model

Total Questions: 15 Complete Sets
Beginner-Friendly: Modular Layout

CRITICAL EXAM HAZARD — Scanner Buffer Pitfall:
Whenever you execute `sc.nextInt()`, `sc.nextDouble()`, or `sc.nextLong()` and need to read a `String` next, always call `sc.nextLine()` immediately after to clear the lingering newline from the input buffer.

Comprehensive Problem Index (15 High-Frequency Exam Types)

Q#	Domain Model	Method 1 Operation	Method 2 Operation
01	Fruits (ID, Name, Price, Rating)	Maximum price filter by rating threshold	Single method exam format
02	Laptop (ID, Brand, OS, Price, Rating)	Count laptops by brand with rating > 3	Filter by OS, sort by ID descending
03	Player (ID, Name, Runs, Type, Match)	Find player with lowest runs by type	Compute average runs by match type
04	Medicine (Name, Batch, Disease, Price)	Extract sorted prices for matching disease	Single method exam format
05	Course (ID, Name, Admin, Fees, Rating)	Calculate average course fees by admin	Sort courses by fees ascending
06	Phone (ID, OS, Brand, Price)	Find price for given brand	Find phone by OS and min price
07	Book (ID, Title, Author, Price, Pages)	Find book with maximum pages by author	Count books priced above threshold
08	Employee (ID, Name, Branch, Rating, Salary)	Count employees by branch with rating >= 4	Find second highest salary employee
09	Movie (Name, Company, Genre, Budget, Rating)	Find average budget for given genre	Filter movies by rating, sort by budget desc
10	AutonomousCar (ID, Brand, Tests, Passed, Env)	Update test details by environment	Find car with second highest pass rate
11	Inventory (ID, MaxQty, CurQty, Threshold)	Filter items needing refill (cur <= thresh)	Count items by threshold category
12	Hotel (ID, Name, DOB, Rooms, Wifi, Total)	Count rooms booked by month of booking	Search hotel with 2nd highest bill
13	NavalVessel (ID, Name, Voyages, Mode, Class)	Find average voyages for given vessel class	Find vessel with highest voyages by mode
14	Sim (ID, Name, Balance, Rate, Circle)	Transfer balance / circle change	Sort sims by balance descending
15	Footwear (ID, Name, Type, Price)	Count footwears by type	Find 2nd highest priced footwear by name

Problem 01: Fruits Management System (Rating & Max Price)

Attributes: `int fruitId`, `String fruitName`, `int price`, `int rating`.
Method: `findMaximumPriceByRating(Fruits[] arr, int targetRating)` — Returns the maximum-priced `Fruits` object with rating > targetRating. If none match, return `null`.
Main: Read 4 fruits, target rating. Print ID or `"No such Fruit"`.

CONSOLE STANDARD INPUT

```
555
Apple
200
5
777
Orange
150
4
333
Banana
100
4
888
Avocado
250
4
3
```

CONSOLE STANDARD OUTPUT

```
888
```

Explanation: Fruits with rating > 3 are Apple (200), Orange (150), Banana (100), and Avocado (250). Avocado has the maximum price (250), so ID 888 is printed.

```java
import java.util.Scanner;

public class Solution {
    // 1. STATIC LOGIC FUNCTION (Implemented First)
    public static Fruits findMaximumPriceByRating(Fruits[] arr, int targetRating) {
        Fruits maxFruit = null;
        for (Fruits f : arr) {
            if (f.getRating() > targetRating) {
                if (maxFruit == null || f.getPrice() > maxFruit.getPrice()) maxFruit = f;
            }
        }
        return maxFruit;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Fruits[] fruits = new Fruits[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine(); // Clear buffer
            String name = sc.nextLine();
            int price = sc.nextInt();
            int rating = sc.nextInt(); sc.nextLine(); // Clear buffer
            fruits[i] = new Fruits(id, name, price, rating);
        }
        int targetRating = sc.nextInt();
        Fruits res = findMaximumPriceByRating(fruits, targetRating);
        if (res != null) System.out.println(res.getFruitId());
        else System.out.println("No such Fruit");
        sc.close();
    }
}

class Fruits {
    private int fruitId, price, rating;
    private String fruitName;
    public Fruits(int fruitId, String fruitName, int price, int rating) {
        this.fruitId = fruitId; this.fruitName = fruitName; this.price = price; this.rating = rating;
    }
    public int getFruitId() { return fruitId; }
    public int getPrice() { return price; }
    public int getRating() { return rating; }
}
```

Problem 02: Laptop Management (Brand Count & Descending Sort)

Attributes: `int laptopId`, `String brand`, `String osType`, `double price`, `int rating`.
Method 1: `countOfLaptopsByBrand(Laptop[] arr, String brand)` — Count laptops matching brand with rating > 3.
Method 2: `searchLaptopByOsType(Laptop[] arr, String os)` — Return laptops matching OS sorted by ID descending.

CONSOLE STANDARD INPUT

```
101
Dell
Windows
55000
4
102
Dell
Ubuntu
45000
3
103
HP
Windows
62000
5
104
Dell
Windows
72000
5
Dell
Windows
```

CONSOLE STANDARD OUTPUT

```
2
104
72000.0
101
55000.0
103
62000.0
```

Explanation: 2 Dell laptops have rating > 3. Windows laptops are 104, 101, and 103, printed in descending order of ID.

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int countOfLaptopsByBrand(Laptop[] arr, String b) {
        int c = 0;
        for (Laptop l : arr) if (l.getBrand().equalsIgnoreCase(b) && l.getRating() > 3) c++;
        return c;
    }

    public static Laptop[] searchLaptopByOsType(Laptop[] arr, String os) {
        List<Laptop> list = new ArrayList<>();
        for (Laptop l : arr) if (l.getOsType().equalsIgnoreCase(os)) list.add(l);
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getLaptopId() - a.getLaptopId());
        return list.toArray(new Laptop[0]);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Laptop[] laptops = new Laptop[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String brand = sc.nextLine(); String os = sc.nextLine();
            double price = sc.nextDouble(); int rating = sc.nextInt(); sc.nextLine();
            laptops[i] = new Laptop(id, brand, os, price, rating);
        }
        String brand = sc.nextLine(); String os = sc.nextLine();
        int count = countOfLaptopsByBrand(laptops, brand);
        if (count > 0) System.out.println(count);
        else System.out.println("The given brand is not available");

        Laptop[] res = searchLaptopByOsType(laptops, os);
        if (res != null) {
            for (Laptop lp : res) { System.out.println(lp.getLaptopId()); System.out.println(lp.getPrice()); }
        } else System.out.println("The given OS is not available");
        sc.close();
    }
}

class Laptop {
    private int laptopId, rating; private String brand, osType; private double price;
    public Laptop(int id, String b, String os, double p, int r) {
        this.laptopId = id; this.brand = b; this.osType = os; this.price = p; this.rating = r;
    }
```

Problem 03: Player Analytics (Lowest Runs & Match Average)

Attributes: `int playerId`, `String playerName`, `int runs`, `String playerType`, `String matchType`.
Method 1: `findPlayerWithLowestRuns(Player[] arr, String type)` — Lowest runs for given playerType.
Method 2: `findAverageRunsByMatchType(Player[] arr, String match)` — Integer average runs for matchType.

CONSOLE STANDARD INPUT

```
11
Sachin
18000
Batsman
ODI
12
Zaheer
1200
Bowler
ODI
13
Virat
13000
Batsman
ODI
14
Dhoni
10500
Batsman
T20
Batsman
ODI
```

CONSOLE STANDARD OUTPUT

```
14
Dhoni
10733
```

Explanation: Lowest runs for Batsman is Dhoni (10500). ODI total runs = 18000 + 1200 + 13000 = 32200 / 3 = 10733.

```java
import java.util.Scanner;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static Player findPlayerWithLowestRuns(Player[] arr, String t) {
        Player minP = null;
        for (Player p : arr) {
            if (p.getPlayerType().equalsIgnoreCase(t)) {
                if (minP == null || p.getRuns() < minP.getRuns()) minP = p;
            }
        }
        return minP;
    }

    public static int findAverageRunsByMatchType(Player[] arr, String m) {
        int sum = 0, count = 0;
        for (Player p : arr) {
            if (p.getMatchType().equalsIgnoreCase(m)) { sum += p.getRuns(); count++; }
        }
        return count > 0 ? (sum / count) : 0;
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Player[] players = new Player[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); int runs = sc.nextInt(); sc.nextLine();
            String pType = sc.nextLine(); String mType = sc.nextLine();
            players[i] = new Player(id, name, runs, pType, mType);
        }
        String pType = sc.nextLine(); String mType = sc.nextLine();
        Player low = findPlayerWithLowestRuns(players, pType);
        if (low != null) { System.out.println(low.getPlayerId()); System.out.println(low.getPlayerName()); }
        else System.out.println("No such player");

        int avg = findAverageRunsByMatchType(players, mType);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No such match type");
        sc.close();
    }
}

class Player {
    private int playerId, runs; private String playerName, playerType, matchType;
```

Problem 04: Medicine Inventory (Filter & Price Array Sort)

Attributes: `String medicineName`, `String batchNo`, `String disease`, `int price`.
Method: `getMedicineByDisease(Medicine[] arr, String disease)` — Return integer array of prices for matching disease sorted in ascending order. If no match, return `null`.
Main: Read 4 medicines, target disease. Print each price or `"No Medicine found"`.

CONSOLE STANDARD INPUT

```
Dolo650
B12
Fever
30
Paracetamol
B14
Fever
20
Cetrizine
B19
Allergy
45
Azithral
B22
Fever
120
Fever
```

CONSOLE STANDARD OUTPUT

```
20
30
120
```

Explanation: Fever medicines are Paracetamol (20), Dolo650 (30), and Azithral (120), sorted in ascending order of price.

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int[] getMedicineByDisease(Medicine[] arr, String disease) {
        List<Integer> list = new ArrayList<>();
        for (Medicine m : arr) {
            if (m.getDisease().equalsIgnoreCase(disease)) list.add(m.getPrice());
        }
        if (list.isEmpty()) return null;
        Collections.sort(list);
        int[] ans = new int[list.size()];
        for (int i = 0; i < list.size(); i++) ans[i] = list.get(i);
        return ans;
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Medicine[] meds = new Medicine[4];
        for (int i = 0; i < 4; i++) {
            String name = sc.nextLine(); String batch = sc.nextLine();
            String disease = sc.nextLine(); int price = sc.nextInt(); sc.nextLine();
            meds[i] = new Medicine(name, batch, disease, price);
        }
        String target = sc.nextLine();
        int[] res = getMedicineByDisease(meds, target);
        if (res != null) {
            for (int p : res) System.out.println(p);
        } else System.out.println("No Medicine found");
        sc.close();
    }
}

class Medicine {
    private String medicineName, batchNo, disease; private int price;
    public Medicine(String n, String b, String d, int p) {
        this.medicineName = n; this.batchNo = b; this.disease = d; this.price = p;
    }
    public String getDisease() { return disease; }
    public int getPrice() { return price; }
}
```

Problem 05: Course Administration (Fee Average & Ascending Sort)

Attributes: `int courseId`, `String courseName`, `String courseAdmin`, `int fee`, `int rating`.
Method 1: `findAvgFeeByAdmin(Course[] arr, String admin)` — Integer average fee for given admin.
Method 2: `sortCourseByFee(Course[] arr, int ratingThreshold)` — Courses with rating <= threshold sorted by fee ascending.

CONSOLE STANDARD INPUT

```
101
Java
John
15000
4
102
Python
John
12000
5
103
SQL
Alex
8000
3
104
AWS
John
20000
3
John
4
```

CONSOLE STANDARD OUTPUT

```
15666
SQL
Java
AWS
```

Explanation: John's average fee = (15000+12000+20000)/3 = 15666. Rating <= 4 sorted: SQL(8k), Java(15k), AWS(20k).

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int findAvgFeeByAdmin(Course[] arr, String a) {
        int sum = 0, count = 0;
        for (Course c : arr) if (c.getCourseAdmin().equalsIgnoreCase(a)) { sum += c.getFee(); count++; }
        return count > 0 ? (sum / count) : 0;
    }

    public static Course[] sortCourseByFee(Course[] arr, int r) {
        List<Course> list = new ArrayList<>();
        for (Course c : arr) if (c.getRating() <= r) list.add(c);
        if (list.isEmpty()) return null;
        list.sort((a, b) -> a.getFee() - b.getFee());
        return list.toArray(new Course[0]);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Course[] courses = new Course[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); String admin = sc.nextLine();
            int fee = sc.nextInt(); int rating = sc.nextInt(); sc.nextLine();
            courses[i] = new Course(id, name, admin, fee, rating);
        }
        String admin = sc.nextLine(); int rating = sc.nextInt();
        int avg = findAvgFeeByAdmin(courses, admin);
        if (avg > 0) System.out.println(avg); else System.out.println("No Such Courses");

        Course[] res = sortCourseByFee(courses, rating);
        if (res != null) {
            for (Course c : res) System.out.println(c.getCourseName());
        } else System.out.println("No certificate found");
        sc.close();
    }
}

class Course {
    private int courseId, fee, rating; private String courseName, courseAdmin;
    public Course(int id, String n, String a, int f, int r) {
        this.courseId = id; this.courseName = n; this.courseAdmin = a; this.fee = f; this.rating = r;
    }
```

Problem 06: Phone Store (Price Query & Minimum Price by OS)

Attributes: `int phoneId, String os, String brand, int price.`
Method 1: `findPriceForGivenBrand(Phone[] arr, String brand)` — Return total sum for matching brand.
Method 2: `getPhoneIdBasedOnOs(Phone[] arr, String os)` — Return phone with price >= 50000 having lowest ID.

CONSOLE STANDARD INPUT

```
111
iOS
Apple
70000
112
Android
Samsung
40000
113
Android
Google
60000
114
Android
OnePlus
52000
Samsung
Android
```

CONSOLE STANDARD OUTPUT

```
40000
113
```

Explanation: Samsung total = 40000. Android phones with price >= 50000 are 113 and 114. Lowest ID is 113.

```java
import java.util.Scanner;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int findPriceForGivenBrand(Phone[] arr, String b) {
        int total = 0;
        for (Phone p : arr) if (p.getBrand().equalsIgnoreCase(b)) total += p.getPrice();
        return total;
    }

    public static Phone getPhoneIdBasedOnOs(Phone[] arr, String os) {
        Phone match = null;
        for (Phone p : arr) {
            if (p.getOs().equalsIgnoreCase(os) && p.getPrice() >= 50000) {
                if (match == null || p.getPhoneId() < match.getPhoneId()) match = p;
            }
        }
        return match;
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Phone[] phones = new Phone[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String os = sc.nextLine(); String brand = sc.nextLine();
            int price = sc.nextInt(); sc.nextLine();
            phones[i] = new Phone(id, os, brand, price);
        }
        String brand = sc.nextLine(); String os = sc.nextLine();
        int sum = findPriceForGivenBrand(phones, brand);
        if (sum > 0) System.out.println(sum); else System.out.println("The given Brand is not available");

        Phone p = getPhoneIdBasedOnOs(phones, os);
        if (p != null) System.out.println(p.getPhoneId());
        else System.out.println("No phones are available with specified os and price range");
        sc.close();
    }
}

class Phone {
    private int phoneId, price; private String os, brand;
    public Phone(int id, String os, String b, int p) {
        this.phoneId = id; this.os = os; this.brand = b; this.price = p;
    }
    public int getPhoneId() { return phoneId; }
    public String getOs() { return os; }
    public String getBrand() { return brand; }
    public int getPrice() { return price; }
```

Problem 07: Library Catalog (Max Pages & Price Count)

Attributes: `int id`, `String title`, `String author`, `double price`, `int pages`.
Method 1: `findBookWithMaximumPages(Book[] arr, String author)` — Return Book by author with maximum pages.
Method 2: `countBooksAbovePrice(Book[] arr, double minPrice)` — Return count of books with price > minPrice.

CONSOLE STANDARD INPUT

```
1
Clean Code
Robert
450.0
464
2
Clean Agile
Robert
350.0
240
3
Refactoring
Martin
550.0
448
4
UML Distilled
Martin
300.0
192
Robert
300.0
```

CONSOLE STANDARD OUTPUT

```
Clean Code
464
3
```

Explanation: Robert's book with max pages is Clean Code (464). Exactly 3 books have price > 300.0.

```java
import java.util.Scanner;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static Book findBookWithMaximumPages(Book[] arr, String author) {
        Book maxB = null;
        for (Book b : arr) {
            if (b.getAuthor().equalsIgnoreCase(author)) {
                if (maxB == null || b.getPages() > maxB.getPages()) maxB = b;
            }
        }
        return maxB;
    }

    public static int countBooksAbovePrice(Book[] arr, double p) {
        int c = 0;
        for (Book b : arr) if (b.getPrice() > p) c++;
        return c;
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] books = new Book[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String title = sc.nextLine(); String author = sc.nextLine();
            double price = sc.nextDouble(); int pages = sc.nextInt(); sc.nextLine();
            books[i] = new Book(id, title, author, price, pages);
        }
        String author = sc.nextLine(); double minPrice = sc.nextDouble();
        Book b = findBookWithMaximumPages(books, author);
        if (b != null) { System.out.println(b.getTitle()); System.out.println(b.getPages()); }
        else System.out.println("No book found for given author");

        int count = countBooksAbovePrice(books, minPrice);
        if (count > 0) System.out.println(count);
        else System.out.println("No books above given price");
        sc.close();
    }
}

class Book {
    private int id, pages; private String title, author; private double price;
    public Book(int id, String t, String a, double p, int pg) {
        this.id = id; this.title = t; this.author = a; this.price = p; this.pages = pg;
```

Problem 08: Workforce Roster (Branch Count & 2nd Highest Salary)

Attributes: `int empId`, `String name`, `String branch`, `double rating`, `double salary`.
Method 1: `countEmployeesByBranch(Employee[] arr, String branch)` — Count employees with rating >= 4.0.
Method 2: `findSecondHighestSalary(Employee[] arr)` — Employee with 2nd highest salary across all branches.

CONSOLE STANDARD INPUT

```
101
Amit
Kolkata
4.5
65000
102
Riya
Delhi
3.8
55000
103
Raj
Kolkata
4.2
72000
104
Tina
Mumbai
4.8
85000
Kolkata
```

CONSOLE STANDARD OUTPUT

```
2
103
Raj
72000.0
```

Explanation: Kolkata employees with rating >= 4.0 = 2. Salaries sorted desc: 85k, 72k, 65k, 55k. 2nd highest is Raj (72000.0).

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int countEmployeesByBranch(Employee[] arr, String b) {
        int c = 0;
        for (Employee e : arr) if (e.getBranch().equalsIgnoreCase(b) && e.getRating() >= 4.0) c++;
        return c;
    }

    public static Employee findSecondHighestSalary(Employee[] arr) {
        if (arr == null || arr.length < 2) return null;
        List<Employee> list = new ArrayList<>(Arrays.asList(arr));
        list.sort((a, b) -> Double.compare(b.getSalary(), a.getSalary()));
        return list.get(1);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Employee[] emps = new Employee[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); String branch = sc.nextLine();
            double rating = sc.nextDouble(); double salary = sc.nextDouble(); sc.nextLine();
            emps[i] = new Employee(id, name, branch, rating, salary);
        }
        String branch = sc.nextLine();
        int count = countEmployeesByBranch(emps, branch);
        if (count > 0) System.out.println(count); else System.out.println("No employees found");

        Employee sec = findSecondHighestSalary(emps);
        if (sec != null) {
            System.out.println(sec.getEmpId()); System.out.println(sec.getName()); System.out.println(sec.getSala
        } else System.out.println("Not enough employees");
        sc.close();
    }
}

class Employee {
    private int empId; private String name, branch; private double rating, salary;
    public Employee(int id, String n, String b, double r, double s) {
        this.empId = id; this.name = n; this.branch = b; this.rating = r; this.salary = s;
    }
    public int getEmpId() { return empId; }
    public String getName() { return name; }
    public String getBranch() { return branch; }
```

Problem 09: Cinema Production (Genre Average & Budget Sort)

Attributes: `String movieName`, `String company`, `String genre`, `int budget`, `int rating`.
Method 1: `findAvgBudgetByGenre(Movie[] arr, String genre)` — Compute integer average budget for given genre.
Method 2: `getMoviesByRating(Movie[] arr, int minRating)` — Movies with rating >= minRating, sorted by budget descending.

CONSOLE STANDARD INPUT

```
Avatar
Fox
SciFi
250
5
Inception
WB
SciFi
160
5
Titanic
Fox
Romance
200
4
Interstellar
WB
SciFi
165
5
SciFi
5
```

CONSOLE STANDARD OUTPUT

```
191
Avatar
Interstellar
Inception
```

Explanation: Average SciFi budget = (250+160+165)/3 = 191. Rating 5 movies sorted by budget descending.

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int findAvgBudgetByGenre(Movie[] arr, String g) {
        int sum = 0, count = 0;
        for (Movie m : arr) if (m.getGenre().equalsIgnoreCase(g)) { sum += m.getBudget(); count++; }
        return count > 0 ? (sum / count) : 0;
    }

    public static Movie[] getMoviesByRating(Movie[] arr, int r) {
        List<Movie> list = new ArrayList<>();
        for (Movie m : arr) if (m.getRating() >= r) list.add(m);
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getBudget() - a.getBudget());
        return list.toArray(new Movie[0]);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Movie[] movies = new Movie[4];
        for (int i = 0; i < 4; i++) {
            String name = sc.nextLine(); String comp = sc.nextLine();
            String genre = sc.nextLine(); int budget = sc.nextInt();
            int rating = sc.nextInt(); sc.nextLine();
            movies[i] = new Movie(name, comp, genre, budget, rating);
        }
        String searchGenre = sc.nextLine(); int searchRating = sc.nextInt();
        int avg = findAvgBudgetByGenre(movies, searchGenre);
        if (avg > 0) System.out.println(avg); else System.out.println("No movies in given genre");

        Movie[] res = getMoviesByRating(movies, searchRating);
        if (res != null) {
            for (Movie m : res) System.out.println(m.getMovieName());
        } else System.out.println("No movies found");
        sc.close();
    }
}

class Movie {
    private String movieName, company, genre; private int budget, rating;
    public Movie(String n, String c, String g, int b, int r) {
        this.movieName = n; this.company = c; this.genre = g; this.budget = b; this.rating = r;
    }
```

Problem 10: Autonomous Car (Pass Calculation & Grade Assignment)

Attributes: `int carId`, `String brand`, `int noOfTestsConducted`, `int noOfTestsPassed`, `String environment`.
Method 1: `findTestPassedByEnv(AutonomousCar[] arr, String env)` — Sum tests passed in environment.
Method 2: `updateCarGrade(AutonomousCar[] arr, String brand)` — Return car of brand with rating = (passed * 100)/conducted.

CONSOLE STANDARD INPUT

```
100
Tesla
1000
850
Urban
101
Waymo
500
450
Highway
102
Cruise
600
400
Urban
103
Baidu
800
600
Desert
Urban
Tesla
```

CONSOLE STANDARD OUTPUT

```
1250
Tesla::A1
```

Explanation: Urban tests passed = 850 + 400 = 1250. Tesla pass percentage = (850*100)/1000 = 85% >= 80, so grade is A1.

```java
import java.util.Scanner;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int findTestPassedByEnv(AutonomousCar[] arr, String env) {
        int sum = 0;
        for (AutonomousCar c : arr) if (c.getEnvironment().equalsIgnoreCase(env)) sum += c.getNoOfTestsPassed();
        return sum;
    }

    public static AutonomousCar updateCarGrade(AutonomousCar[] arr, String b) {
        for (AutonomousCar c : arr) if (c.getBrand().equalsIgnoreCase(b)) return c;
        return null;
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AutonomousCar[] cars = new AutonomousCar[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String brand = sc.nextLine(); int tests = sc.nextInt();
            int passed = sc.nextInt(); sc.nextLine();
            String env = sc.nextLine();
            cars[i] = new AutonomousCar(id, brand, tests, passed, env);
        }
        String searchEnv = sc.nextLine(); String searchBrand = sc.nextLine();
        int totalPassed = findTestPassedByEnv(cars, searchEnv);
        if (totalPassed > 0) System.out.println(totalPassed);
        else System.out.println("There are no tests passed in this particular environment");

        AutonomousCar car = updateCarGrade(cars, searchBrand);
        if (car != null) {
            int rating = (car.getNoOfTestsPassed() * 100) / car.getNoOfTestsConducted();
            String grade = rating >= 80 ? "A1" : "B2";
            System.out.println(car.getBrand() + "::" + grade);
        } else System.out.println("No Car is available with the specified brand");
        sc.close();
    }
}

class AutonomousCar {
    private int carId, noOfTestsConducted, noOfTestsPassed; private String brand, environment;
    public AutonomousCar(int id, String b, int t, int p, String e) {
        this.carId = id; this.brand = b; this.noOfTestsConducted = t; this.noOfTestsPassed = p; this.environment
```

Problem 11: Warehouse Inventory (Replenishment Threshold Check)

Attributes: `int inventoryId`, `int maxQuantity`, `int currentQuantity`, `int threshold`.
Method: `replenish(Inventory[] arr, int limit)` — Return array of `Inventory` objects whose threshold <= limit.
Main: Print ID followed by `"Critical Filling"` if current <= 50, else `"Moderate Filling"`.

CONSOLE STANDARD INPUT

```
1
100
50
50
2
200
120
80
3
150
30
40
4
80
20
30
50
```

CONSOLE STANDARD OUTPUT

```
1 Critical Filling
3 Critical Filling
4 Critical Filling
```

Explanation: Items with threshold <= 50 are 1, 3, and 4. All three have currentQuantity <= 50, so labeled Critical Filling.

```
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static Inventory[] replenish(Inventory[] arr, int limit) {
        List<Inventory> list = new ArrayList<>();
        for (Inventory inv : arr) if (inv.getThreshold() <= limit) list.add(inv);
        return list.isEmpty() ? null : list.toArray(new Inventory[0]);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Inventory[] items = new Inventory[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); int maxQ = sc.nextInt();
            int curQ = sc.nextInt(); int th = sc.nextInt(); sc.nextLine();
            items[i] = new Inventory(id, maxQ, curQ, th);
        }
        int limit = sc.nextInt();
        Inventory[] res = replenish(items, limit);
        if (res != null) {
            for (Inventory inv : res) {
                if (inv.getThreshold() >= 75) System.out.println(inv.getInventoryId() + " Non-Critical Filling");
                else if (inv.getCurrentQuantity() <= 50) System.out.println(inv.getInventoryId() + " Critical Fil
                else System.out.println(inv.getInventoryId() + " Moderate Filling");
            }
        }
        sc.close();
    }
}

class Inventory {
    private int inventoryId, maxQuantity, currentQuantity, threshold;
    public Inventory(int id, int maxQ, int curQ, int th) {
        this.inventoryId = id; this.maxQuantity = maxQ; this.currentQuantity = curQ; this.threshold = th;
    }
    public int getInventoryId() { return inventoryId; }
    public int getCurrentQuantity() { return currentQuantity; }
    public int getThreshold() { return threshold; }
}
```

Problem 12: Hotel Booking (Month Parsing & 2nd Highest Bill)

Attributes: `int hotelId`, `String hotelName`, `String dateOfBooking`, `int noOfRoomsBooked`, `String wifiFacility`, `double totalBill`.
Method 1: `noOfRoomsBookedInGivenMonth(Hotel[] arr, String month)` — Sum rooms booked in month.
Method 2: `searchHotelByWifiOption(Hotel[] arr, String wifi)` — Hotel with 2nd highest totalBill matching wifi.

CONSOLE STANDARD INPUT

```
101
Hyatt
12-May-2022
3
Yes
15000
102
Taj
15-May-2022
5
Yes
35000
103
Radisson
20-Jun-2022
2
Yes
25000
104
Marriott
01-May-2022
4
No
18000
May
Yes
```

CONSOLE STANDARD OUTPUT

```
12
103
```

Explanation: May rooms booked: 3 + 5 + 4 = 12. Wifi 'Yes' bills: Taj(35k), Radisson(25k), Hyatt(15k). 2nd highest ID is 103.

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int noOfRoomsBookedInGivenMonth(Hotel[] arr, String m) {
        int count = 0;
        for (Hotel h : arr) if (h.getDateOfBooking().contains(m)) count += h.getNoOfRoomsBooked();
        return count;
    }

    public static Hotel searchHotelByWifiOption(Hotel[] arr, String w) {
        List<Hotel> list = new ArrayList<>();
        for (Hotel h : arr) if (h.getWifiFacility().equalsIgnoreCase(w)) list.add(h);
        if (list.size() < 2) return null;
        list.sort((a, b) -> Double.compare(b.getTotalBill(), a.getTotalBill()));
        return list.get(1);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Hotel[] hotels = new Hotel[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); String dob = sc.nextLine();
            int rooms = sc.nextInt(); sc.nextLine();
            String wifi = sc.nextLine(); double bill = sc.nextDouble(); sc.nextLine();
            hotels[i] = new Hotel(id, name, dob, rooms, wifi, bill);
        }
        String month = sc.nextLine(); String wifi = sc.nextLine();
        int totalRooms = noOfRoomsBookedInGivenMonth(hotels, month);
        if (totalRooms > 0) System.out.println(totalRooms);
        else System.out.println("No rooms booked in the given month");

        Hotel sec = searchHotelByWifiOption(hotels, wifi);
        if (sec != null) System.out.println(sec.getHotelId());
        else System.out.println("No such option available");
        sc.close();
    }
}
```

Problem 13: Maritime Registry (Average Voyages & Highest by Mode)

Attributes: `int vesselId`, `String vesselName`, `int noOfVoyagesPlanned`, `int noOfVoyagesCompleted`, `String purpose`.
Method 1: `findAvgVoyagesByPct(NavalVessel[] arr, int percentage)` — Average completed voyages for completion % >= percentage.
Method 2: `findVesselByGrade(NavalVessel[] arr, String purpose)` — Return vessel of purpose with highest completed voyages.

CONSOLE STANDARD INPUT

```
111
Vikrant
10
9
Defense
112
INS Arihant
12
10
Defense
113
CargoX
8
4
Commercial
114
Patrol1
5
5
Defense
80
Defense
```

CONSOLE STANDARD OUTPUT

```
8
INS Arihant%Defense
```

Explanation: Vessels >= 80%: Vikrant(90%), Arihant(83%), Patrol1(100%). Average = (9+10+5)/3 = 8. Highest completed is Arihant (10).

```java
import java.util.Scanner;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int findAvgVoyagesByPct(NavalVessel[] arr, int pct) {
        int sum = 0, count = 0;
        for (NavalVessel v : arr) {
            int curPct = (v.getNoOfVoyagesCompleted() * 100) / v.getNoOfVoyagesPlanned();
            if (curPct >= pct) { sum += v.getNoOfVoyagesCompleted(); count++; }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static NavalVessel findVesselByGrade(NavalVessel[] arr, String p) {
        NavalVessel best = null;
        for (NavalVessel v : arr) {
            if (v.getPurpose().equalsIgnoreCase(p)) {
                if (best == null || v.getNoOfVoyagesCompleted() > best.getNoOfVoyagesCompleted()) best = v;
            }
        }
        return best;
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        NavalVessel[] vessels = new NavalVessel[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); int planned = sc.nextInt();
            int completed = sc.nextInt(); sc.nextLine();
            String purpose = sc.nextLine();
            vessels[i] = new NavalVessel(id, name, planned, completed, purpose);
        }
        int pct = sc.nextInt(); sc.nextLine(); String searchPurpose = sc.nextLine();
        int avg = findAvgVoyagesByPct(vessels, pct);
        if (avg > 0) System.out.println(avg); else System.out.println("No vessels found");

        NavalVessel best = findVesselByGrade(vessels, searchPurpose);
        if (best != null) System.out.println(best.getVesselName() + "%" + best.getPurpose());
        else System.out.println("No Naval Vessel is available");
        sc.close();
```

Problem 14: Telecom SIM (Circle Modification & Rate Sorting)

Attributes: `int simId`, `String customerName`, `double balance`, `double ratePerSecond`, `String circle`.
Method: `transferCircle(Sim[] arr, String circle1, String circle2)` — Return sims matching circle1 with circle updated to circle2, sorted in descending order of `ratePerSecond`.
Main: Read 4 sims, circle1, circle2. Print ID, customerName, circle, rate.

CONSOLE STANDARD INPUT

```
1
Rahul
150.0
1.2
Kolkata
2
Rohan
200.0
1.8
Delhi
3
Priya
300.0
1.5
Kolkata
4
Ankit
120.0
0.8
Kolkata
Kolkata
Mumbai
```

CONSOLE STANDARD OUTPUT

```
3 Priya Mumbai 1.5
1 Rahul Mumbai 1.2
4 Ankit Mumbai 0.8
```

Explanation: Kolkata SIMs transferred to Mumbai and sorted descending by rate: Priya (1.5), Rahul (1.2), and Ankit (0.8).

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static Sim[] transferCircle(Sim[] arr, String c1, String c2) {
        List<Sim> list = new ArrayList<>();
        for (Sim s : arr) {
            if (s.getCircle().equalsIgnoreCase(c1)) {
                s.setCircle(c2);
                list.add(s);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(b.getRatePerSecond(), a.getRatePerSecond()));
        return list.toArray(new Sim[0]);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Sim[] sims = new Sim[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); double bal = sc.nextDouble();
            double rate = sc.nextDouble(); sc.nextLine();
            String circle = sc.nextLine();
            sims[i] = new Sim(id, name, bal, rate, circle);
        }
        String c1 = sc.nextLine(); String c2 = sc.nextLine();
        Sim[] res = transferCircle(sims, c1, c2);
        if (res != null) {
            for (Sim s : res) {
                System.out.println(s.getSimId() + " " + s.getCustomerName() + " " + s.getCircle() + " " + s.getRa
            }
        }
        sc.close();
    }
}

class Sim {
    private int simId; private String customerName, circle; private double balance, ratePerSecond;
    public Sim(int id, String n, double b, double r, String c) {
        this.simId = id; this.customerName = n; this.balance = b; this.ratePerSecond = r; this.circle = c;
    }
    public int getSimId() { return simId; }
```

Problem 15: Footwear Store (Count by Type & 2nd Highest Price)

Attributes: `int footwearId`, `String footwearName`, `String footwearType`, `int price`.
Method 1: `getCountByType(Footwear[] arr, String type)` — Return count of footwear for given type.
Method 2: `getSecondHighestPriceByBrand(Footwear[] arr, String name)` — Return footwear with 2nd highest price for name.

CONSOLE STANDARD INPUT

```
100
Nike
Running
5000
101
Adidas
Running
4500
102
Nike
Casual
3500
103
Nike
Running
7000
Running
Nike
```

CONSOLE STANDARD OUTPUT

```
3
100
Nike
5000
```

Explanation: Running shoes count = 3. Nike footwear prices: 7000, 5000, 3500. 2nd highest price is 5000 (ID 100).

```java
import java.util.*;

public class Solution {
    // 1. STATIC METHODS FIRST
    public static int getCountByType(Footwear[] arr, String t) {
        int c = 0;
        for (Footwear f : arr) if (f.getFootwearType().equalsIgnoreCase(t)) c++;
        return c;
    }

    public static Footwear getSecondHighestPriceByBrand(Footwear[] arr, String n) {
        List<Footwear> list = new ArrayList<>();
        for (Footwear f : arr) if (f.getFootwearName().equalsIgnoreCase(n)) list.add(f);
        if (list.size() < 2) return null;
        list.sort((a, b) -> b.getPrice() - a.getPrice());
        return list.get(1);
    }

    // 2. MAIN RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Footwear[] items = new Footwear[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt(); sc.nextLine();
            String name = sc.nextLine(); String type = sc.nextLine();
            int price = sc.nextInt(); sc.nextLine();
            items[i] = new Footwear(id, name, type, price);
        }
        String searchType = sc.nextLine(); String searchName = sc.nextLine();
        int count = getCountByType(items, searchType);
        if (count > 0) System.out.println(count);
        else System.out.println("Footwear not available");

        Footwear sec = getSecondHighestPriceByBrand(items, searchName);
        if (sec != null) {
            System.out.println(sec.getFootwearId());
            System.out.println(sec.getFootwearName());
            System.out.println(sec.getPrice());
        } else System.out.println("Brand not available");
        sc.close();
    }
}

class Footwear {
    private int footwearId, price; private String footwearName, footwearType;
    public Footwear(int id, String n, String t, int p) {
        this.footwearId = id; this.footwearName = n; this.footwearType = t; this.price = p;
    }
    public int getFootwearId() { return footwearId; }
```